Następny etap: galeria

Propozycja do wspólnej decyzji. Nie jest to jeszcze zaimplementowana funkcja.

Cel użytkownika

Po wybraniu folderu widzę miniatury jak w eksploratorze. Klikam zdjęcie, oglądam je w większym podglądzie i widzę jego metadane. W kolejnych etapach obok pojawią się OCR, tagi i propozycje do zatwierdzenia.

1. Ustalić widok
Siatka miniatur + zaznaczenie + panel jednego zdjęcia.

↓

2. Zmierzyć aktualny czas
Skan, odczyt obrazu, hash, baza, API i React osobno.

↓

3. Pobierać katalog porcjami
Nie zwracać i nie renderować wszystkich rekordów naraz.

↓

4. Bezpieczny podgląd
Java sprawdza dostęp do obrazu; źródło pozostaje read-only.

↓

5. Dopiero potem optymalizować
Zmieniać najwolniejszy zmierzony etap i testować wynik.

OCR warto dołożyć, gdy jest już miejsce, w którym użytkownik zobaczy oryginał i oceni wynik analizy. Rozpoznawanie obrazu przed podglądem trudniej byłoby zweryfikować.

700 plików: gdzie znika czas?

To hipotezy wynikające z kodu, nie wyniki pomiaru Twojego komputera.

Etap	Obecne działanie i możliwy koszt
Odczyt obrazu	ImageIO.read dekoduje każdy obsługiwany plik.
Hash	Files.readAllBytes ponownie czyta całą zawartość i potrzebuje pamięci na jej bajty.
Baza	CatalogUpdater dla każdego zaakceptowanego pliku szuka rekordu i go zapisuje.
Katalog	CatalogReader pobiera całą listę folderu.
Widok	React renderuje wszystkie wiersze wyniku i katalogu.

Pomiar przed decyzją

Porównać pierwszy i drugi skan małego folderu oraz 700 plików. Dla każdego etapu zapisać czas i liczbę plików. Dopiero wtedy wybrać między strumieniowym hashowaniem, pomijaniem niezmienionych plików, zmniejszeniem liczby zapytań, ograniczoną równoległością i innymi zmianami.

Stronicowanie poprawia pobieranie i wyświetlanie katalogu, ale samo nie przyspieszy czytania zdjęć podczas skanu.

Która struktura danych?

Szybsza struktura ma sens tylko wtedy, gdy przyspiesza operację, która naprawdę dominuje.

Struktura	Ocena dla ScreenSort teraz
ArrayList	Dobry wybór do dopisywania wyników i iteracji. Już używana.
LinkedList	Nie przyspiesza skanu; indeksowanie przechodzi przez węzły, węzły kosztują pamięć.
HashSet	Przydatny do sprawdzania obecności elementu, gdy pojawi się taka potrzeba.
HashMap	Przydatna do wyszukiwania po kluczu w pamięci; wymaga dodatkowej pamięci.
TreeSet / TreeMap	Przydatne do utrzymywania porządku; obecny katalog sortuje baza.
Wyszukiwanie binarne	Wymaga listy posortowanej po tym samym kluczu; nie zastępuje indeksu SQL.

W PostgreSQL już jest unikalność pary (folder, ścieżka względna) oraz indeks SHA-256. HashMap w Javie nie zastąpi tych indeksów. Dla wielu wyników bardziej użyteczne będzie porcjowanie odpowiedzi API.

Dalszy przebieg

Galeria i podgląd → OCR i analiza → sugestia nazwy/kategorii/tagów → ręczna korekta i zatwierdzenie → zweryfikowana kopia do osobnego folderu. Oryginałów nie zmieniamy.

Pełny opis: docs/nastepny-etap-i-wydajnosc.md. Decyzje bezpieczeństwa: docs/adr/ADR-001-copy-first.md.